

1. Locator là gì? Tại sao locator bị hỏng? Ví dụ test case fail?

Locator là “địa chỉ” để Selenium tìm một phần tử HTML trên trang – Ví dụ By.ID, By.XPATH, By.CSS_SELECTOR, data-testid, name,...

Locator bị hỏng khi developer thay đổi UI mà không thông báo cho QA. Cụ thể trong hệ thống này, Mutation_engine.py đang mô phỏng tình huống này. Ví dụ từ v1->v2

("data-testid", "login-email"): ("data-testid", "email-field"),

("data-testid", "login-password"): ("data-testid", "password-field"),

Ví dụ test case bị hỏng:

Test script viết cho v1:

driver.find_element(By.XPATH, '//[@data-testid="btn-login"]').click()*

Sau khi UI chuyển sang v2, developer đổi thành:

data-testid="login-submit-btn"

➔ Selenium ném NoSuchElementException ➔ test FAIL

Exception_interceptor xác nhận NoSuchElementException là dấu hiệu “UI đã thay đổi ➔ Kích hoạt self-healing pipeline”

2. Ngoài locator hỏng, test UI còn fail vì nguyên nhân gì khác? Hệ thống có xử lý không?

Hệ thống được xây dựng có thể xử lý 5 loại exception khác nhau trong Exception_interceptor.py.

Exception	Nguyên nhân	Chiến lược xử lý
NoSuchElementException	Locator hỏng	TRIGGER_HEALING ➔ full healing pipeline
StaleElementReferenceException	DOM re-render (React/Vue)	RETRY_FIND ➔ thử lại 3 lần
TimeoutException	Trang load chậm	WAIT_AND_RETRY ➔ chờ thêm 10s
ElementNotInteractableException	Element bị ẩn/ ngoài viewport	SCROLL_INTO_VIEW ➔ JS scroll
ElementClickInterceptedException	Overlay/ modal che mất	JS_CLICK ➔ click bằng JavaScript

➔ Hệ thống có thể xử lý được cả 5 nguyên nhân phổ biến nhất, không chỉ riêng locator bị hỏng.

3. Tại sao không bảo developer dùng data-testid mãi và không bao giờ thay đổi?

Trong thực tế, việc duy trì không thay đổi các locator như data-testid là không khả thi do sự thay đổi liên tục của UI, framework, và yêu cầu hệ thống (ví dụ accessibility hoặc refactoring). Điều này tạo ra sự phụ thuộc chặt chẽ giữa test script và implementation của giao diện, dẫn đến chi phí bảo trì cao và độ ổn định thấp.

- ➔ Self healing test automation đóng vai trò như một lớp trung gian, cho phép hệ thống kiểm thử thích ứng với các thay đổi nhỏ của UI bằng cách suy luận và lựa chọn locator thay thế phù hợp. Nhờ đó, hệ thống giảm thiểu lỗi giả (false negatives), tăng độ ổn định của pipeline CI/CD và giảm đáng kể chi phí bảo trì test.

4. Flaky test là gì? Quan hệ với locator hỏng?

Flaky test là khi test chạy cho ra kết quả không nhất quán – lúc pass, lúc fail – dù code không đổi.

Mối quan hệ: Locator hỏng là một trong những nguyên nhân gây ra flaky test:

- Khi UI load bất đồng bộ (React render chậm), locator có thể tìm thấy lần này nhưng không lần sau → StaleElementReferenceException → flaky
- Khi data-testid đổi ở một số môi trường (staging) nhưng chưa deploy lên test → Locator hỏng ngẫu nhiên theo môi trường.

5. Ngưỡng 7.5/5.0 có ý nghĩa gì? Nếu đặt ngưỡng quá cao thì sẽ có hậu quả gì?

Ý nghĩa:

- ≥ 0.75 → AUTO_FIX: Hệ thống đủ tự tin, tự động thay locator và tiếp tục test.
- $0.50-0.75$ → SUGGEST: Hệ thống tìm được ứng viên nhưng không chắc chắn, chỉ gợi ý.
- < 0.50 → FAILED: Không tìm được phần tử tin cậy, báo lỗi.

Hậu quả nếu đặt sai:

- Ngưỡng AUTO quá cao (ví dụ 0.95): Rất ít healing thành công, hầu hết rơi vào SUGGEST/FAILED → hệ thống gần như vô dụng.
- Ngưỡng AUTO quá thấp (ví dụ 0.30): Heal sai phần tử → test âm thầm chạy sai → nguy hiểm hơn là fail vì tester không biết kết quả sai.

6. SelfHealingDriver hoạt động như thế nào? Tester cần phải thay đổi gì trong test script khi dùng bảo vệ hệ thống?

Luồng hệ thống:

Đầu tiên là chạy như bình thường | → pass → lưu snapshot → trả về element

|→ fail → ExceptionInterceptor phân loại

|--- STALE_ELEMENT → retry_find()

|--- TIMEOUT → wait_and_find()

|--- NOT_INTERACABLE → scroll_into_view()

|--- CLICK_INTERCEPTED → js_click()

|--- UNKNOWN → raise()

|--- LOCATOR_NOT_FOUND → trigger_healing()

|--- cache hit → dùng locator cũ

|--- cache miss → full healing pipeline

(quét DOM → tính similarity → rank →
lưu KB để học

Tester chỉ cần đổi 2 dòng:

Trước:

```
driver = webdriver.Chrome()
```

```
driver.find_element(By.ID, "btn-login")
```

Sau:

```
driver = SelfHealingDriver(webdriver.Chrome(), test_name="login_test")
```

```
driver.find_element(By.ID, "btn-login", step_name="click_login_btn",  
snapshot=FP["click_login_btn"])
```

Tất cả logic healing diễn ra hoàn toàn tự động bên trong, không ảnh hưởng đến luồng test.

7. Hàm capture_snapshot thu thập dữ liệu thế nào?

Quy trình thu thập :

```
capture_snapshot(driver, element, step_name, ui_version)
```

Nhóm 1: Attribute – Selenium API

- element.tag_name → tag
- element.get_attribute("id") → el_id
- element.get_attribute("class") → classes (split thành list)
- element.get_attribute("name") → name

- element.get_attribute("type") → input_type
- element.get_attribute("placeholder") → placeholder
- element.get_attribute("aria-label") → aria_label
- element.get_attribute("data-testid") → data_testid

Nhóm 2: Semantic – Selenium API

- element.text → text (strip + None nếu rỗng)
- element.get_attribute("role") → role
- element.get_attribute("href") → href
- element.get_attribute("value") → value

Nhóm 3: Structure -JavaScript Executor

- [JS] getXPath(el) → xpath_abs
 - o Hàm đệ quy leo ngược DOM:
 - Nếu el.id → trả ngay '//{tag}[@id="..."]'
 - Nếu là body → '/html/body'
 - Không thì đếm vị trí trong siblings + leo tiếp lên cha
- [JS] arguments[0].parentElement → parent (WebElement)
- Dùng Selenium API đọc tiếp:
 - o parent.tag_name → parent_tag
 - o parent.get_attribute("id") → parent_id
 - o parent.get_attribute("class") → parent_classes
- [JS] Array.from(parent.children).indexOf(el) → sibling_index
 - o parent.children.length → siblings_count
- [JS] Vòng lặp while(el.parentElement){d++} → depth

Nhóm 4: Visual

- [Selenium] element.location → bounding_x, bounding_y
- [Selenium] element.size → bounding_w, bounding_h
- [Selenium] element.is_displayed() → is_visible

- [Selenium] element.is_enabled() → is_enabled
- [JS] window.getComputedStyle().fontSize → computed_font_size
- [JS] window.getComputedStyle().color → computed_color

Nhóm 5: Context

- [Selenium] driver.current_url → page_url
- [Selenium] driver.title → page_title
- [JS] nearby_label
 - o Cách 1: querySelector('label[for="id"]') — label liên kết qua for
 - o Cách 2: leo ngược qua parentElement cho đến khi gặp LABEL (dừng nếu gặp FORM để không leo quá xa)
- [JS] form_context
 - o Leo ngược qua parentElement cho đến khi gặp FORM → trả về el.id || el.className || 'form'

➔ Trả về ElementSnapshot (Tất cả dữ liệu trên)

P/S: Mỗi nhóm đều được bọc trong try/except. Nếu một bước nào fail (phần tử không có cha, JS lỗi, trang đang load...) thì field đó trả về None hoặc giá trị default – Không crash toàn bộ quá trình thu thập.

Tại sao cần dùng JavaScript Executor mà không dùng Selenium API thường?

Vì:

- Selenium không có API trả về Âpth của phần tử. Code dùng hàm JS đệ quy getXPath() tự leo lên DOM từ phần tử đến gốc để xây dựng chuỗi xpath
- Selenium không có API đếm anh em cùng cấp. Code dùng JS Array.from(parent.children).indexOf(el) để phân biệt phần tử đang ở vị trí thứ mấy trong parent
- Selenium không có API đếm số cấp cha. Code dùng JS vòng lặp leo lên qua el.parentElement liên tục cho đến khi không còn cha nữa
- Selenium không có expose window.getComputedStyle(). Code dùng JS để lấy font-size và color thực tế mà browser render (khác với vị trí khai báo trong CSS, vì có thể bị override bởi nhiều rule)
- Selenium không có API tra cứu quan hệ DOM để tìm <label for =”id”> hoặc <form> gần nhất

8. Khi DOM có 500 phần tử, hệ thống tính similarity cho cả 500 không? Nếu không thì lọc ứng viên như nào?

Khi DOM có 500 phần tử thì hệ thống KHÔNG tính similarity cho cả 500 mà hệ thống lọc theo 2 bước trong `find_candidates()`:

B1: Lọc theo tag

```
elements = driver.find_elements(By.TAG_NAME, original.tag)
```

Nếu original là `<button>`, chỉ lấy các `<button>` trong DOM (thường chỉ 5–20 cái). Nếu là `<input>`, chỉ lấy `<input>`

B2: Lọc element ẩn:

```
if not elem.is_displayed(): continue
```

```
if elem.tag_name in ["body", "html", "script", "style"]: continue
```

B3: Threshold filter: Chỉ giữ lại ứng viên có `total_score` ≥ 0.4

9. Hệ thống dùng Machine Learning ở bước nào và để làm gì?

ML được dùng để tối ưu hóa trọng số cho bài toán phân loại ứng viên đúng sai trong quá trình healing.

Dữ liệu đầu vào “`candidate_scores.csv`”: 407 dòng, mỗi dòng là 1 ứng viên DOM với 5 feature scores (attr, sem, struct, visual, ctx) và nhãn `is_correct` (1=ứng viên đúng, 0= ứng viên sai). Phân phối khá mất cân bằng: 327 mẫu âm (80.3%) và 80 mẫu dương (19.7%)

Luồng chạy của ML:

1. Chọn feature: Chỉ dùng 5 similarity scores, bỏ `total_score` để tránh data leakage
2. Split dữ liệu: Dùng `GroupShuffleSplit` theo `healing_event_id` thay vì `train_test_split` thông thường, để đảm bảo một healing event không bị chia ra cả train lẫn test – tránh leakage
3. Train model: Logistic Regression với `class_weight='balanced'` để xử lý mất cân bằng dữ liệu
4. Evaluate:
 - Accuracy: 96.23%
 - Recall: 100%
 - Precision : 80%
 - F1_score: 88.89%
 - ROC-AUC: 99.03%

- Top-1 Success Rate = 100% trên 16 healing event test nghĩa là model luôn xếp ứng viên đúng đầu tiên
 - Top-1 Success Rate = 100% là chỉ số quan trọng nhất. Trong bài toán self-healing, hệ thống không cần dự đoán đúng nhãn cho từng ứng viên riêng lẻ — điều thực sự quan trọng là khi có nhiều ứng viên, ứng viên đúng phải được xếp hạng cao nhất. Kết quả này cho thấy model làm được điều đó trên toàn bộ 16 healing events test.
 - Recall = 100% cũng rất quan trọng. Trong self-healing, bỏ sót ứng viên đúng nguy hiểm hơn nhiều so với chọn nhầm — vì bỏ sót nghĩa là test fail hoàn toàn, còn chọn nhầm ít nhất còn có cơ hội phát hiện sau.
 - Precision = 80% Confusion matrix cho thấy 4 false positive — nghĩa là 4 ứng viên sai bị model dự đoán là đúng. Tuy nhiên điều này không ảnh hưởng đến Top-1 vì model vẫn xếp ứng viên đúng lên đầu.

5. Extract learned weights: Lấy hệ số (coefficients) của model, normalixe thành trọng số để thay thế trọng số cứng trong SimilarityEngine

P/S: ML chưa được tích hợp vào hệ thống, chỉ dùng để tối ưu hệ số riêng rồi thay vào hệ số được tính theo phương pháp SAW và AHP ban đầu để so sánh thực nghiệm.

10. Tại sao dùng Logistic Regression mà không dùng Neural Network hay Deep Learning

Tiêu chí	Logistic Regression	Neural Network	Deep learning
Dữ liệu nhỏ	Hoạt động tốt	Overfit	Overfit
Dễ giải thích	Coefficient = trọng số	Black box – không thể biết feature nào quan trọng hơn	Black box sâu hơn
Tốc độ train	Milliseconds	Phút đến giờ	Chậm
Đưa ra trọng số tối ưu	Trực tiếp	Cần thêm bước	Khó/ không trực tiếp
Yêu cầu tài nguyên	Nhẹ(CPU đủ)	Trung bình	Nặng (GPU thường cần)

➔ Với dữ liệu nhỏ, số feature ít và yêu cầu giải thích rõ ràng, Logistic Regression là lựa chọn tối ưu. Neural Network và Deep Learning không phù hợp do dễ overfitting, khó giải thích và không mang lại cải thiện hiệu năng trong bài toán này

11. Nếu chưa có dữ liệu lịch sử (lần chạy đầu tiên) hệ thống sẽ xử lý như thế nào?

Ở lần chạy đầu tiên khi chưa có dữ liệu lịch sử, hệ thống sẽ không thực hiện self healing mà hoạt động theo cơ chế locator truyền thống. Nếu phần tử được tìm thấy, hệ thống sẽ capture_snapshot

và lưu lại thông qua SnapshotStore.save(), từ đó tạo dữ liệu nền cho các lần chạy sau. Nếu không tìm thấy phần tử thì test sẽ fail như thường. Sau lần chạy đầu tiên, khi đã có snapshot, hệ thống mới có thể sử dụng dữ liệu này để so sánh và kích hoạt cơ chế self healing ở các lần chạy tiếp theo.

12. Hệ thống của bạn có thể chọn sai phần tử không? Khi nào thì chọn sai? Hậu quả là gì?

Hệ thống CÓ THỂ chọn sai, khi:

- Hai phần tử có cùng đặc điểm: VD: trang có 2 button “Submit” trong 2 form khác nhau similarity score gần bằng nhau → hệ thống chọn sai form.
- Score cao nhưng sai ngữ nghĩa: VD: ô input email và ô input search đều có type=”text”, class tương tự, cùng depth → attribute score cao nhưng mục đích hoàn toàn khác nhau
- UI redesign lớn: Element mới không còn liên quan đến snapshot cũ nhưng vô tình có nhiều đặc điểm trùng khớp

Hậu quả:

- Test tiếp tục chạy nhưng tương tác với phần tử sai → kết quả test sai mà không có lỗi rõ ràng
- Đây là dạng false positive nguy hiểm nhất – test báo PASS nhưng thực ra là flow đã broken
- Cụ thể trong lần test contact em đã gặp phải trường hợp này khi chạy test. Hệ thống đã heal nhầm ô input name thành input phone và sau đó khi hệ thống chạy test case thì nó điền tên vào ô số điện thoại rồi điền số điện thoại tiếp ở ô đó làm cho ô tên rỗng → test case đáng ra được dự đoán đúng thì thành sai → test FAIL

13. Nếu developer redesign hoàn toàn giao diện (đổi layout, đổi flow, đổi toàn bộ element) hệ thống có heal được không?

Tình huống	Kết quả
Đổi data-testid, giữ nguyên cấu trúc	Heal được – attribute thay đổi nhưng những phần khác vẫn match
Đổi toàn bộ kiểu locator (data-testid → aria-label)	Heal được – semantic score vẫn cao nếu text/label tương tự
Đổi layout (button chuyển ra chỗ khác so với ban đầu)	Heal được -visual score giảm nhưng những phần khác vẫn match
Đổi toàn bộ: tag, text, vị trí, context,..	Fail – không còn điểm tương đồng nào đủ vượt ngưỡng
Xóa hoàn toàn tính năng (flow mới)	Fail – element thực sự không còn tồn tại

- ➔ self healing xử lý tốt các thay đổi “cosmetic” và “refactor” nhưng không thể thay thế việc cập nhật test script khi có redesign toàn diện

14. Hệ thống heal tự động có nguy hiểm không? trường hợp nào gây hại hơn có lợi?

Hệ thống heal tự động CÓ nguy hiểm trong các trường hợp sau:

1. Che giấu bug thực sự: Nếu developer xóa 1 button đi business logic thay đổi, nhưng hệ thống heal sang một button khác ➔ test pass ➔ bug không được phát hiện ➔ đây là trường hợp nguy hiểm nhất
2. Drift tích lũy (accumulated drift): Healing lần 1 chọn đúng 90%. Snapshot được cập nhật dựa trên element mới này. Healing lần 2 dựa trên snapshot đã drift ➔ Chọn đúng 85%. Sau nhiều vòng hệ thống ngày càng xa rời intent ban đầu của test
3. CI/CD tự động: Trong pipeline CI/CD, nếu hệ thống tự heal âm thầm và toàn bộ test đều pass, bug có thể bị đẩy thẳng lên production. Khi đó lỗi được phát hiện bởi người dùng thay vì QA
4. Ngưỡng auto-fix quá thấp: Nếu similarity score thấp nhưng vẫn cho phép auto-fix thì khả năng chọn nhầm element là rất cao, làm kết quả mất độ tin cậy.